SQL

```
-- create database week1_db;
-- use week1_db;
-- CREATE TABLE Departments (
--     dept_id INT,
--     dept_name VARCHAR(100)
-- );

-- CREATE TABLE Employees (
--     emp_id INT,
--     emp_name VARCHAR(100),
--     dept_id INT,
--     salary DECIMAL(10,2),
--     manager_id INT,
--     join_date DATE
-- );

-- CREATE TABLE Projects (
--     project_id INT,
--     project_name VARCHAR(100),
--     dept_id INT,
--     budget DECIMAL(12,2)
-- );

-- CREATE TABLE EmployeeProjects (
--     emp_id INT,
--     project_id INT,
```

```
--    hours_worked DECIMAL(8,2)
-- );

-- CREATE TABLE Orders (
--    order_id INT,
--    customer_id INT,
--    order_date DATE,
--    amount DECIMAL(12,2),
--    status VARCHAR(20)
-- );




-- INSERT INTO Departments (dept_id, dept_name) VALUES
-- (1, 'Engineering'),
-- (2, 'Human Resources'),
-- (3, 'Finance'),
-- (4, 'Marketing'),
-- (5, 'Sales');

-- NSERT INTO Employees
-- (emp_id, emp_name, dept_id, salary, manager_id, join_date)
-- VALUES
-- (101, 'Rajesh Kumar', 1, 120000, NULL, '2018-06-15'),
-- (102, 'Priya Sharma', 2, 95000, NULL, '2019-03-10'),
-- (103, 'Arjun Reddy', 3, 110000, NULL, '2017-08-20'),
-- (104, 'Sneha Rao', 4, 100000, NULL, '2020-01-12'),
-- (105, 'Vikram Singh', 5, 105000, NULL, '2018-11-05'),
```

```
-- (106, 'Anil Kumar', 1, 75000, 101, '2021-07-19'),
-- (107, 'Meena Devi', 1, 82000, 101, '2022-02-14'),
-- (108, 'Rahul Verma', 1, 68000, 101, '2023-06-01'),

-- (109, 'Kavya Nair', 2, 65000, 102, '2021-09-15'),
-- (110, 'Rohit Das', 2, 60000, 102, '2023-01-20'),

-- (111, 'Neha Patel', 3, 72000, 103, '2022-04-18'),
-- (112, 'Suresh Babu', 3, 68000, 103, '2021-12-10'),

-- (113, 'Aisha Khan', 4, 70000, 104, '2022-08-22'),
-- (114, 'Manoj Gupta', 4, 62000, 104, '2023-05-17'),

-- (115, 'Deepak Joshi', 5, 73000, 105, '2020-10-11'),
-- (116, 'Pooja Mehta', 5, 67000, 105, '2022-11-25');

-- INSERT INTO Projects
-- (project_id, project_name, dept_id, budget)
-- VALUES
-- (201, 'PhotoVault', 1, 500000),
-- (202, 'AI Recommendation System', 1, 750000),
-- (203, 'Employee Management System', 2, 300000),
-- (204, 'Financial Analytics Platform', 3, 600000),
-- (205, 'Digital Marketing Campaign', 4, 250000),
-- (206, 'Sales Automation System', 5, 450000),
-- (207, 'Customer Data Platform', 1, 900000);
```

```
-- INSERT INTO EmployeeProjects
-- (emp_id, project_id, hours_worked)
-- VALUES
-- (101, 201, 120),
-- (106, 201, 180),
-- (107, 201, 160),
-- (108, 201, 140),

-- (101, 202, 100),
-- (107, 202, 210),
-- (108, 202, 190),

-- (102, 203, 80),
-- (109, 203, 170),
-- (110, 203, 150),

-- (103, 204, 100),
-- (111, 204, 220),
-- (112, 204, 180),

-- (104, 205, 90),
-- (113, 205, 200),
-- (114, 205, 175),

-- (105, 206, 110),
-- (115, 206, 230),
-- (116, 206, 195),
```

```
-- (101, 207, 90),
-- (106, 207, 200),
-- (107, 207, 180),
-- (108, 207, 160);

-- INSERT INTO Orders
-- (order_id, customer_id, order_date, amount, status)
-- VALUES
-- (1001, 501, '2026-01-05', 15000, 'Delivered'),
-- (1002, 502, '2026-01-08', 8500, 'Shipped'),
-- (1003, 503, '2026-01-12', 22000, 'Delivered'),
-- (1004, 504, '2026-01-15', 4500, 'Cancelled'),
-- (1005, 501, '2026-02-02', 12000, 'Delivered'),
-- (1006, 505, '2026-02-10', 35000, 'Pending'),
-- (1007, 506, '2026-02-14', 7800, 'Shipped'),
-- (1008, 502, '2026-02-20', 18500, 'Delivered'),
-- (1009, 507, '2026-03-01', 27000, 'Delivered'),
-- (1010, 508, '2026-03-05', 9200, 'Pending'),
-- (1011, 503, '2026-03-10', 15000, 'Shipped'),
-- (1012, 509, '2026-03-15', 42000, 'Delivered'),
-- (1013, 510, '2026-03-20', 6700, 'Cancelled'),
-- (1014, 501, '2026-04-02', 25000, 'Delivered'),
-- (1015, 505, '2026-04-10', 31000, 'Pending');



SELECT * FROM Departments;
SELECT * FROM Employees;
```

SELECT * FROM Projects;

SELECT * FROM EmployeeProjects;

SELECT * FROM Orders;

-- q1

1. Department Salary Summary Show department name, employee count, average, minimum and maximum salary. Include departments with no employees

select d.dept_id,d.dept_name,count(e.emp_id),avg(e.salary),min(e.salary),max(e.salary) from departments d left join employees e on d.dept_id=e.dept_id group by d.dept_id,d.dept_name;

-- q2

2. Second Highest Salary per Department Using a window function, return employee(s) earning the second distinct highest salary in each department

select dept_id,emp_name,salary from (

select dept_id,emp_name,salary,dense_rank() over(partition by dept_id order by salary desc)

as salary_rank from employees) as e where salary_rank=2;

-- q3

3. Employees Above Department Average Find employees earning more than their own department average without hard-coding departments

SELECT emp_id, emp_name, dept_id, salary FROM employees e WHERE salary > (SELECT AVG(e2.salary) FROM employees e2 WHERE e2.dept_id = e.dept_id);

-- q4

4. Manager vs Team Salary For each manager, show direct-report count, total team salary and manager salary; keep managers with at least two reports.

SELECT m.emp_id AS manager_id, m.emp_name AS manager_name, m.salary AS manager_salary, COUNT(e.emp_id) AS direct_report_count, SUM(e.salary) AS

total_team_salary FROM Employees m JOIN Employees e ON m.emp_id = e.manager_id GROUP BY m.emp_id, m.emp_name, m.salary HAVING COUNT(e.emp_id) >= 2 ORDER BY m.emp_id;

-- q5

5. Top Project Contributors For each project, rank employees by total hours and return the top two contributors, handling ties correctly.

select emp_id,project_id,total_hours from (

select emp_id,project_id,sum(hours_worked) as total_hours,

dense_rank() over(partition by project_id order by sum(hours_worked)) as rnk from employeeProjects group by project_id,emp_id) e where rnk <=2 ORDER BY project_id, rnk;

-- q6

6. Departments Without Active Projects Find departments that contain employees but have no matching project

SELECT d.dept_id, d.dept_name FROM Departments d JOIN Employees e ON d.dept_id = e.dept_id

LEFT JOIN Projects p ON d.dept_id = p.dept_id WHERE p.project_id IS NULL GROUP BY d.dept_id, d.dept_name;

-- q7

7. Running Order Revenue For completed orders, show order date, daily revenue and cumulative revenue ordered by date.

SELECT order_date, SUM(amount) AS daily_revenue,SUM(SUM(amount)) OVER (ORDER BY order_date) AS cumulative_revenue

FROM Orders WHERE status = 'Delivered' GROUP BY order_date ORDER BY order_date;

-- q8

8. Customer Order Classification Classify each customer by completed-order value: PLATINUM >=100000, GOLD >=50000, SILVER >=20000, else BRONZE

select customer_id,

```
case
when sum(amount) >=100000 then "PLATINUM"
when sum(amount) >=50000 then "GOLD"
when sum(amount) >=20000 then "SILVER"
else "BRONZE"
end as custmor_clasiifiction
from orders where status="Delivered"  group by customer_id order by customer_id;
-- q9
```

9. Consecutive Monthly Orders Identify customers with at least one order in three consecutive calendar months using CTEs/window functions

```
-- q10
```

10. Salary Gap Analysis For each department, return highest-paid and lowest-paid employee(s) and the salary difference; preserve ties.

```
select dept_id, emp_id, emp_name, salary,
case when salary = highest_salary then 'highest'
when salary = lowest_salary then 'lowest'
end as salary_type,
highest_salary - lowest_salary as salary_difference
from (select dept_id,emp_id,emp_name,salary,max(salary) over (partition by dept_id) as highest_salary,min(salary) over (partition by dept_id) as lowest_salary
from employees) e where salary = highest_salary or salary = lowest_salary order by dept_id, salary desc;
```